Exact-Coverage Transmitter Placement

Design and Analysis of Algorithms — Project Report (Question 8)

Your Name
Roll Number

October 27, 2026

1 Problem Statement

There are m houses along a street, at positions $h_1, \ldots, h_m$ (metres from a fixed origin). Transmitters may be built only at n candidate positions $c_1, \ldots, c_n$. Write $R = 500$ for the transmission range. A transmitter at c_j *reaches* house h_i if $|h_i - c_j| \le R$.

A set $S \subseteq \{1, \ldots, n\}$ of candidates is *valid* if every house is reached by *exactly one* transmitter of S (at least one for coverage, at most one to avoid interference). The goal is to find a valid set of minimum size, or to report that no valid set exists.

Implementation. The full Python implementation, tests and README are available at: `https://github.com/YOUR-USERNAME/YOUR-REPOSITORY`.

2 A Case Where No Solution Exists

Example 1. Let $R = 500$, houses at 0, 600, 1200, and candidate locations 300 and 900.

- The candidate at 300 reaches the houses at 0 and 600 (distances 300 and 300), but not 1200 (distance 900).
- The candidate at 900 reaches the houses at 600 and 1200 (distances 300 and 300), but not 0.

The house at 0 is reached only by the candidate 300, and the house at 1200 only by the candidate 900, so both must be chosen. Then the house at 600 is reached by two transmitters, violating the exactness requirement. Choosing only one candidate leaves a house uncovered. The four subsets of candidates are $\emptyset$, $\{300\}$, $\{900\}$, $\{300, 900\}$, and none is valid. Hence this instance has no solution.

3 Structure of the Problem

Sort the houses so that $h_1 \le h_2 \le \cdots \le h_m$ and index them $0, \ldots, m-1$ if convenient.

Lemma 1 (Contiguity)**.** *For every candidate c, the set of houses it reaches is a block of consecutive houses in the sorted order, i.e. it equals $\{h_{l+1}, \ldots, h_r\}$ for some $0 \le l \le r \le m$.*

Proof. House h is reached by c iff $h \in [c - R,\ c + R]$. Let l be the number of houses with $h < c - R$ and r the number of houses with $h \le c + R$. Since the list is sorted, the houses with $h < c - R$ are exactly $h_1, \ldots, h_l$ and those with $h \le c + R$ are exactly $h_1, \ldots, h_r$. Therefore the houses with $c - R \le h \le c + R$ are exactly $h_{l+1}, \ldots, h_r$. (Houses with equal positions are either all inside or all outside the interval, so ties cause no trouble.) □

Denote the block of candidate j by $B_j = (l_j, r_j]$, meaning houses $l_j + 1, \ldots, r_j$. Candidates with $l_j = r_j$ reach no house; they never help and are discarded.

Lemma 2 (Reformulation)**.** *A set S of candidates (each with nonempty block) is valid if and only if the blocks $\{B_j : j \in S\}$ form a partition of $\{1, \ldots, m\}$, that is, they are pairwise disjoint and their union is $\{1, \ldots, m\}$.*

Proof. House i is reached by exactly one chosen transmitter iff i lies in exactly one chosen block. Requiring this for all i is precisely disjointness together with full union. □

Consequently, the problem is: *partition $\{1, \ldots, m\}$ into the fewest blocks taken from the available family $\{B_j\}$.* Two candidates with identical blocks cannot both be chosen (they would overlap), and an empty block can be dropped without affecting validity, so restricting to nonempty blocks loses no minimum solution.

4 Algorithm

Let $f(i)$ denote the minimum number of blocks from the family that partition $\{1, \ldots, i\}$, with $f(0) = 0$ and $f(i) = \infty$ if no such partition exists. The answer to the problem is $f(m)$; the value ∞ means that no solution exists.

Lemma 3 (Recurrence)**.** *For $1 \le i \le m$,*

$$f(i) \;=\; \min\big\{\, f(l) + 1 \;:\; \textit{there is a candidate } j \textit{ with } B_j = (l, i] \,\big\},$$

where the minimum of an empty set is ∞.

Proof. Consider any partition of $\{1, \ldots, i\}$ into family blocks. House i lies in exactly one block of the partition, say $B_j = (l, r]$. Since all blocks lie inside $\{1, \ldots, i\}$, we have $r \le i$, and since $i \in B_j$ we get $r \ge i$; hence $r = i$ and $B_j = (l, i]$. The remaining blocks then partition $\{1, \ldots, l\}$ (they are disjoint from B_j and cover everything except B_j). So a partition of $\{1, \ldots, i\}$ with k blocks yields a partition of $\{1, \ldots, l\}$ with $k - 1$ blocks, giving $k - 1 \ge f(l)$, i.e. $k \ge f(l) + 1$ for some admissible l. This shows $f(i) \ge$ the right-hand side.

Conversely, if $B_j = (l, i]$ is in the family and $f(l) < \infty$, take an optimal partition of $\{1, \ldots, l\}$ and add B_j. Since B_j is disjoint from $\{1, \ldots, l\}$, this is a partition of $\{1, \ldots, i\}$ with $f(l) + 1$ blocks, so $f(i) \le f(l) + 1$. This shows $f(i) \le$ the right-hand side. If no admissible block ends at i, no partition exists and $f(i) = \infty$. □

4.1 Pseudocode

1. Sort the houses.
2. For each candidate c_j, find by binary search $l_j = \#\{h < c_j - R\}$ and $r_j = \#\{h \le c_j + R\}$. If $l_j < r_j$, record the triple (r_j, l_j, j).
3. Sort the triples by r_j (increasing).
4. Set $\mathrm{dp}[0] = 0$ and $\mathrm{dp}[i] = \infty$ for $i \ge 1$.
5. For each triple (r, l, j) in sorted order: if $\mathrm{dp}[l] + 1 < \mathrm{dp}[r]$, set $\mathrm{dp}[r] = \mathrm{dp}[l] + 1$ and $\mathrm{parent}[r] = (l, j)$.
6. If $\mathrm{dp}[m] = \infty$, report No Solution.
7. Otherwise, set $i = m$; while $i > 0$: let $(l, j) = \mathrm{parent}[i]$, output candidate j, and set $i = l$.

5 Correctness

Theorem 1. *After step 5, $\mathrm{dp}[i] = f(i)$ for all $0 \le i \le m$. Consequently the algorithm reports* No Solution *exactly when no valid placement exists, and otherwise outputs a valid placement of minimum size.*

Proof. Order of processing. Every recorded block has $l < r$. When the triple (r, l, j) is processed, every triple with right end l has already been processed, because $l < r$ and triples are sorted by right end. We claim that at the moment a triple with right end r is processed, $\mathrm{dp}[l]$ already equals $f(l)$. This follows by strong induction on the right end: $\mathrm{dp}[0] = 0 = f(0)$ from the start and is never changed (updates only write to positions $r \geq 1$). For $l \geq 1$, by Lemma 3 $f(l)$ is the minimum of $f(l') + 1$ over blocks ending at l, and every such block is processed before any triple with right end greater than l; by the induction hypothesis each of these updates used the correct $f(l')$. Hence $\mathrm{dp}[l] = f(l)$ once all triples with right end l are done (and $\mathrm{dp}[l]$ stays ∞ if none exist, matching $f(l) = \infty$).

Final values. By the same argument and Lemma 3, after all triples have been processed, $\mathrm{dp}[i] = \min\{f(l) + 1\}$ over blocks $(l, i]$, which equals $f(i)$. In particular $\mathrm{dp}[m] = f(m)$.

Existence. By Lemma 2, a valid placement exists iff $\{1, \ldots, m\}$ can be partitioned into family blocks, i.e. iff $f(m) < \infty$. So the algorithm's answer for existence is correct.

Reconstruction. Whenever $\mathrm{parent}[r]$ is set, it records the block achieving the current minimum, and the final value of $\mathrm{parent}[r] = (l, j)$ satisfies $\mathrm{dp}[r] = \mathrm{dp}[l] + 1$. Following the pointers from m gives blocks $(l_1, m], (l_2, l_1], \ldots, (0, l_k]$ that are consecutive and hence form a partition of $\{1, \ldots, m\}$ into exactly $\mathrm{dp}[m] - 0 = k$ blocks, so the output is a valid placement of size $f(m)$, which is minimum. □

6 Complexity Analysis

Step	Work	Time
Sort houses	comparison sort	$O(m \log m)$
Compute l_j, r_j for all candidates	$2n$ binary searches on m items	$O(n \log m)$
Sort the triples	comparison sort of $\leq n$ items	$O(n \log n)$
DP sweep	one $O(1)$ update per triple	$O(n)$
Reconstruction	at most $\min(m, n)$ pointer steps	$O(\min(m, n))$
Total		$O(m \log m + n \log n + n \log m)$

The term $n \log m$ is absorbed by the others: if $m \leq n$ then $n \log m \leq n \log n$, and if $m > n$ then $n \log m \leq m \log m$. Hence the running time simplifies to

$$O\big((m + n) \log(m + n)\big).$$

The space used is $O(m + n)$: the sorted houses, the triples, and the arrays dp and parent.

Optimality remark. Reading the input already takes $\Omega(m + n)$ time, and the algorithm is within a logarithmic factor of this, which is due only to sorting. (If the inputs are given pre-sorted, the binary searches can be replaced by two-pointer sweeps and the algorithm runs in $O(m + n)$.)

7 Implementation Notes

The implementation (`transmitter_placement/solver.py`) follows the pseudocode directly.

- Distances are compared with $\leq$ (inclusive), using `bisect_left` for l_j and `bisect_right` for r_j.

- The inputs may be unsorted and may contain duplicates; houses are sorted internally and candidates keep their original indices.
- The function returns `None` when no valid placement exists, and otherwise the count, the original candidate indices, and their positions.
- A separate function `is_valid` checks the "exactly one transmitter per house" condition and is used in the tests.

8 Testing

The test-suite (`tests/test_solver.py`, run with `python -m unittest discover -s tests`) contains

- hand-made cases: a single house, no houses, an uncovered house, boundary distance exactly R, unsorted input, duplicate positions, preferring fewer transmitters, and the no-solution example of Section 2;
- 1500 random small instances compared against an exhaustive search over all subsets of candidates. Both the existence answer and the minimum size agree on every instance, and the validity of every returned placement is verified. The random generator produces both solvable and unsolvable instances in substantial numbers.

9 Conclusion

Because each transmitter reaches a contiguous block of houses, exact coverage with the fewest transmitters reduces to partitioning a sorted line into blocks from a given family, which a simple dynamic programme solves optimally in $O((m+n)\log(m+n))$ time and $O(m+n)$ space, and which also decides correctly whether any solution exists.